

CS675

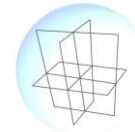
Computer Graphics

The OpenGL API

Parag Chaudhuri



Department of Computer Science and Engineering
IIT Bombay



ViGIL
VISION, GRAPHICS,
IMAGING & LEARNING

Introducing OpenGL

What is OpenGL?

- Open Graphics Library
- *API* to specify geometric objects in 2D/3D and to control how they are *rendered* into the framebuffer.
- A software interface to graphics hardware.
- Cross language, cross platform, open source
- Alternatives – Direct3D (Microsoft)

OpenGL Variants

- OpenGL ES
 - For hand-held devices and embedded environments
- WebGL
 - Javascript implementation of ES
 - Runs on most recent browsers

Bleeding Edge Graphics Programming

- Vulkan (cross platform)
- Metal (apple only)
- DirectX DXR (microsoft only)

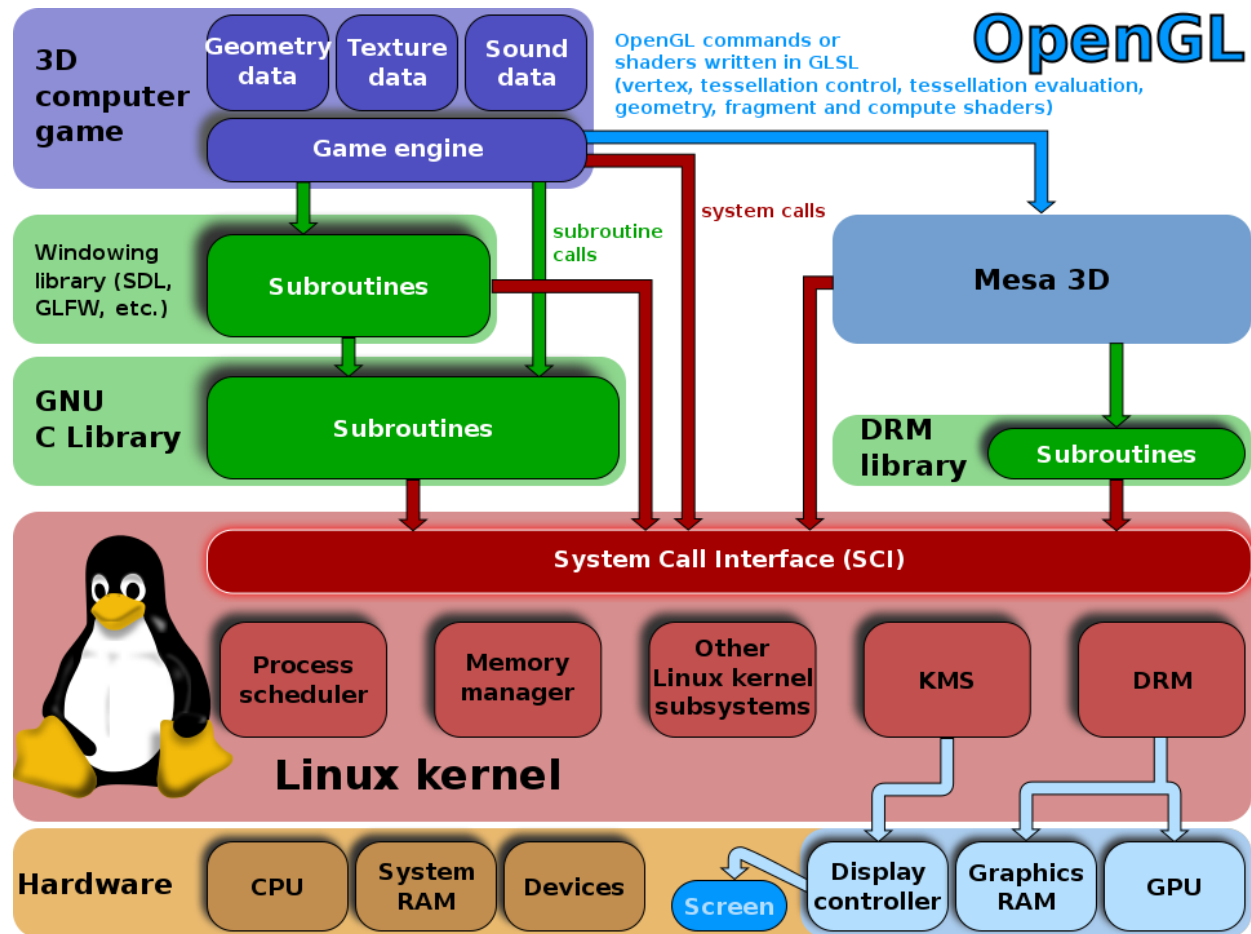


<https://devblogs.nvidia.com/vulkan-raytracing/>

Shading Languages

- Shading Languages
 - OpenGL Shading Language (GLSL)
 - High Level Shading Language (HLSL, DirectX)
 - CUDA
 - OpenCL (Open Compute Language)
 - CUDA and OpenCL are/have become more than shading languages and are now used for general purpose compute on the GPU.
 - Standard Portable Intermediate Representation (SPIR-V) – intermediate language for device independent shaders

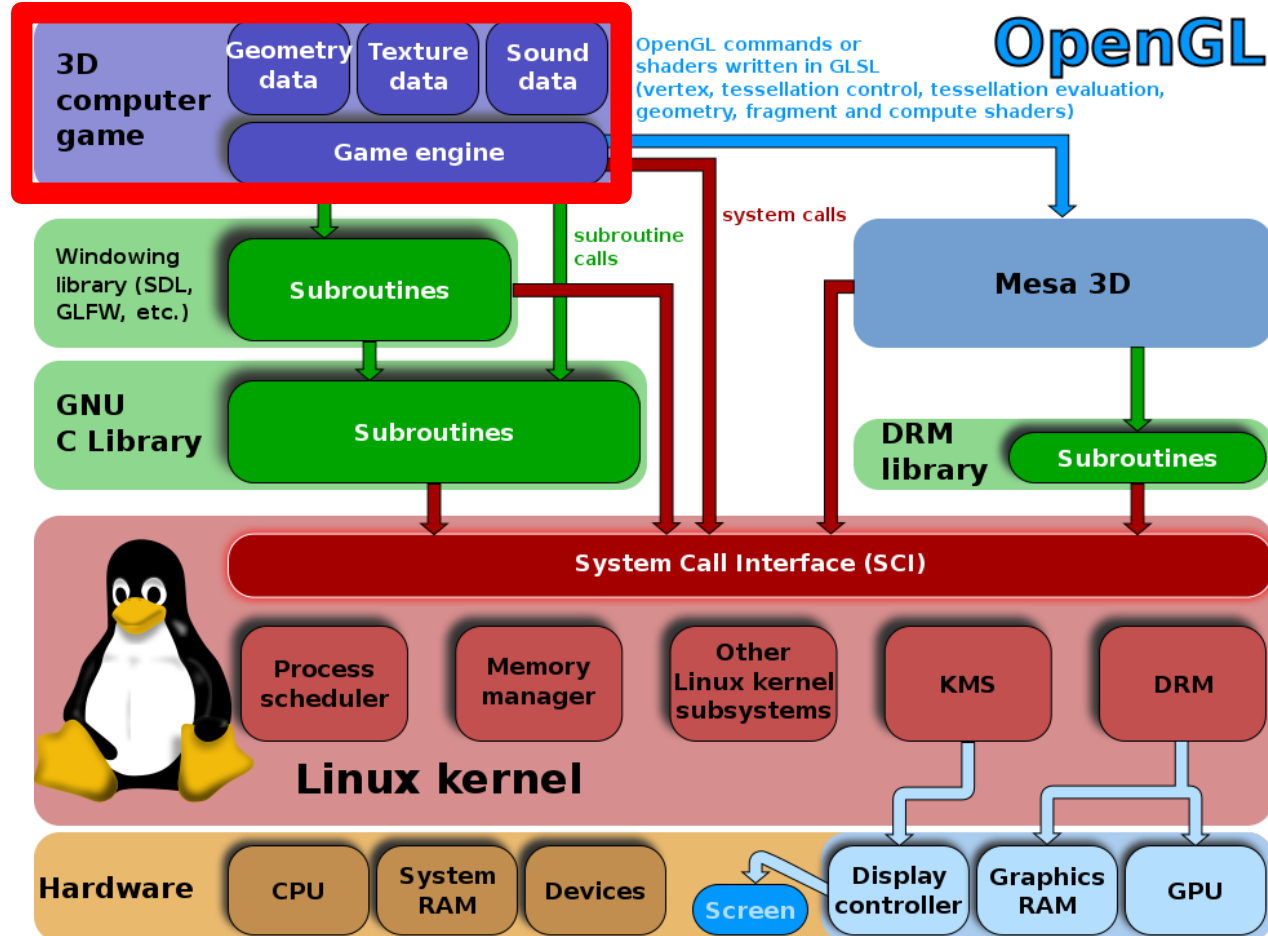
The Graphics HW/SW Boundary



The Graphics HW/SW Boundary

Application Program
(User Level)

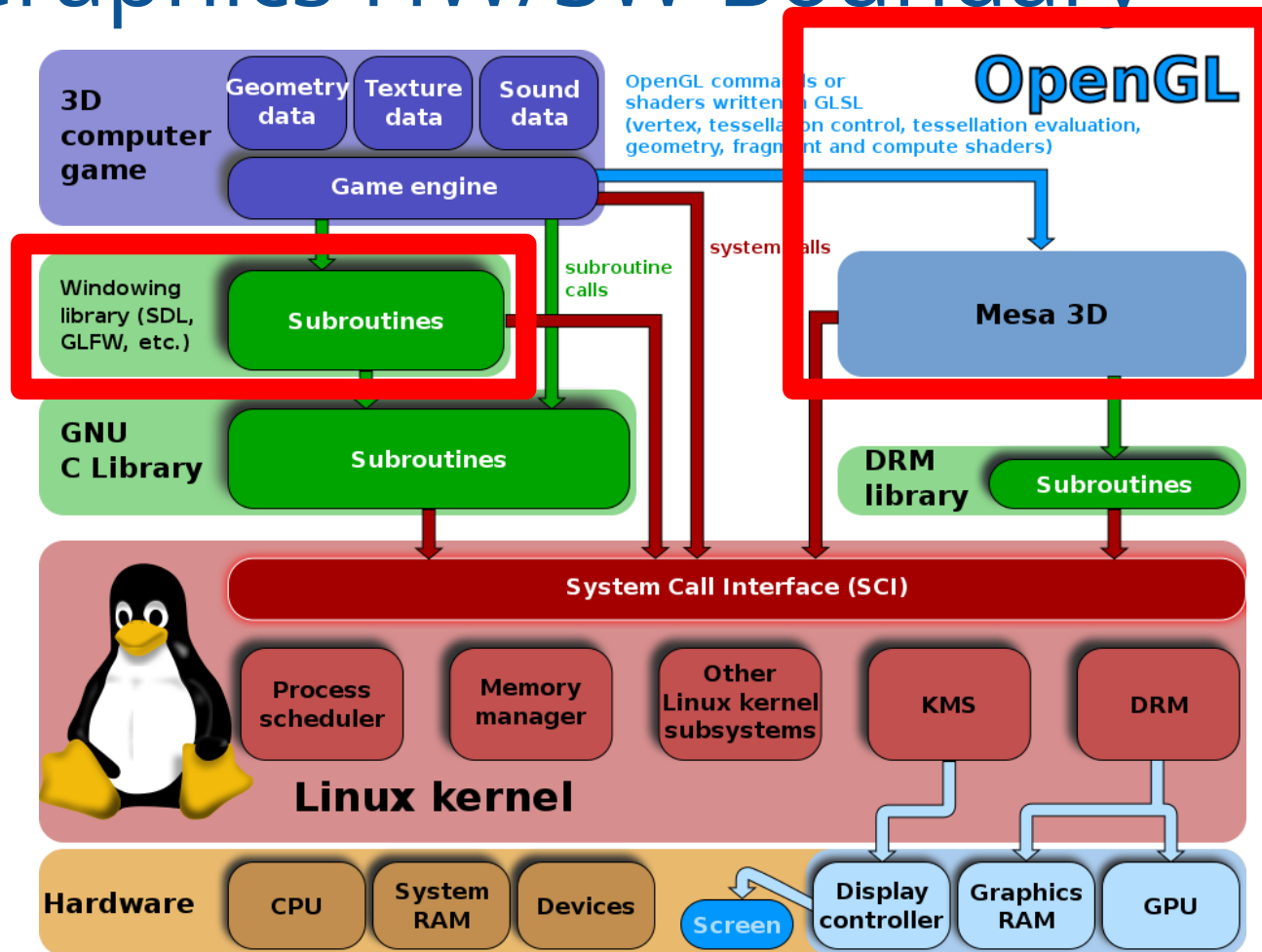
Uses the
OpenGL API in a
high level
programming
language.



The Graphics HW/SW Boundary

Application also uses a windowing library, like GLFW or GLUT to access windowing functions and input devices like the keyboard and mouse.

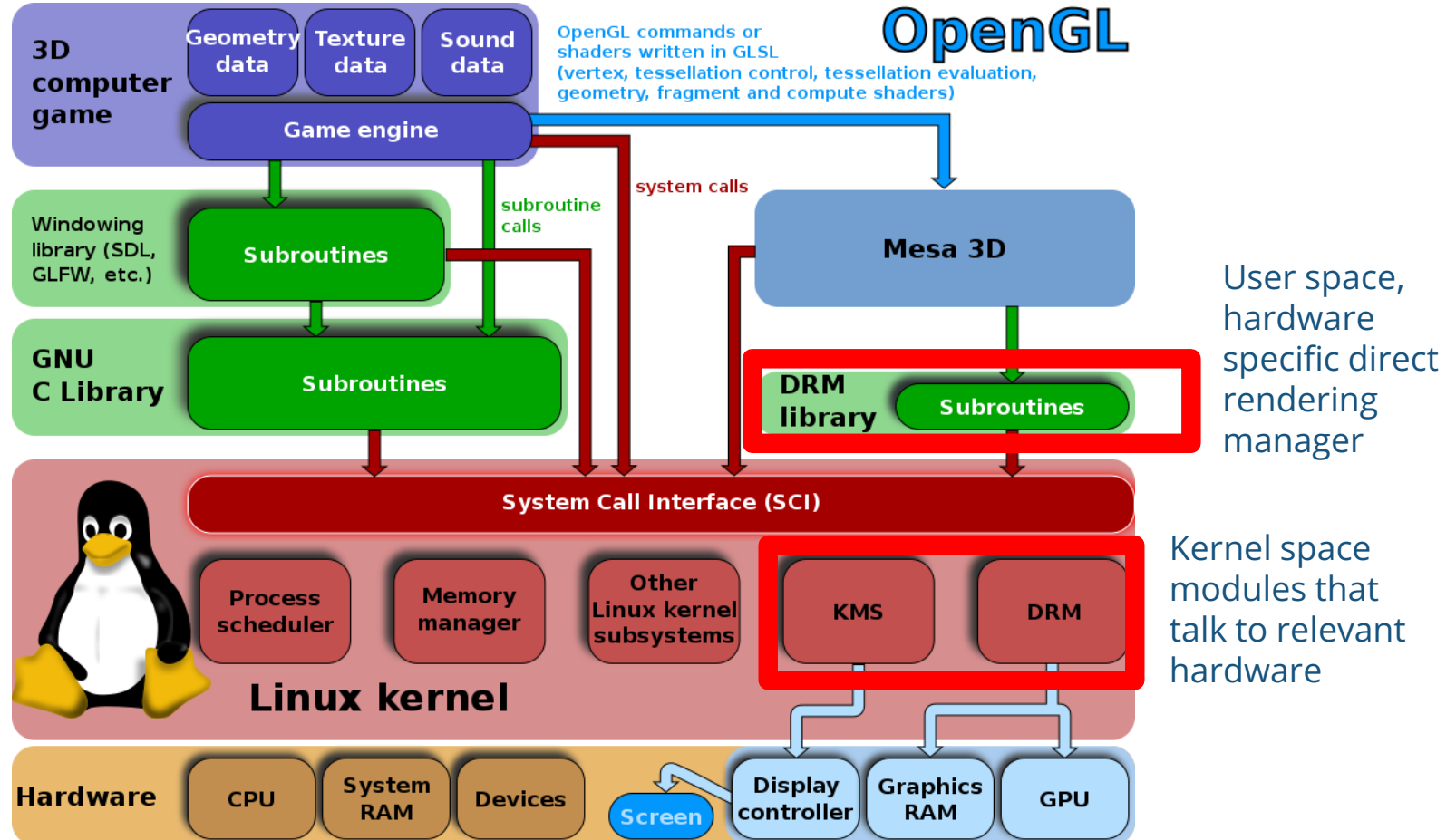
These implement platform independent wrapper APIs over platform specific window managers

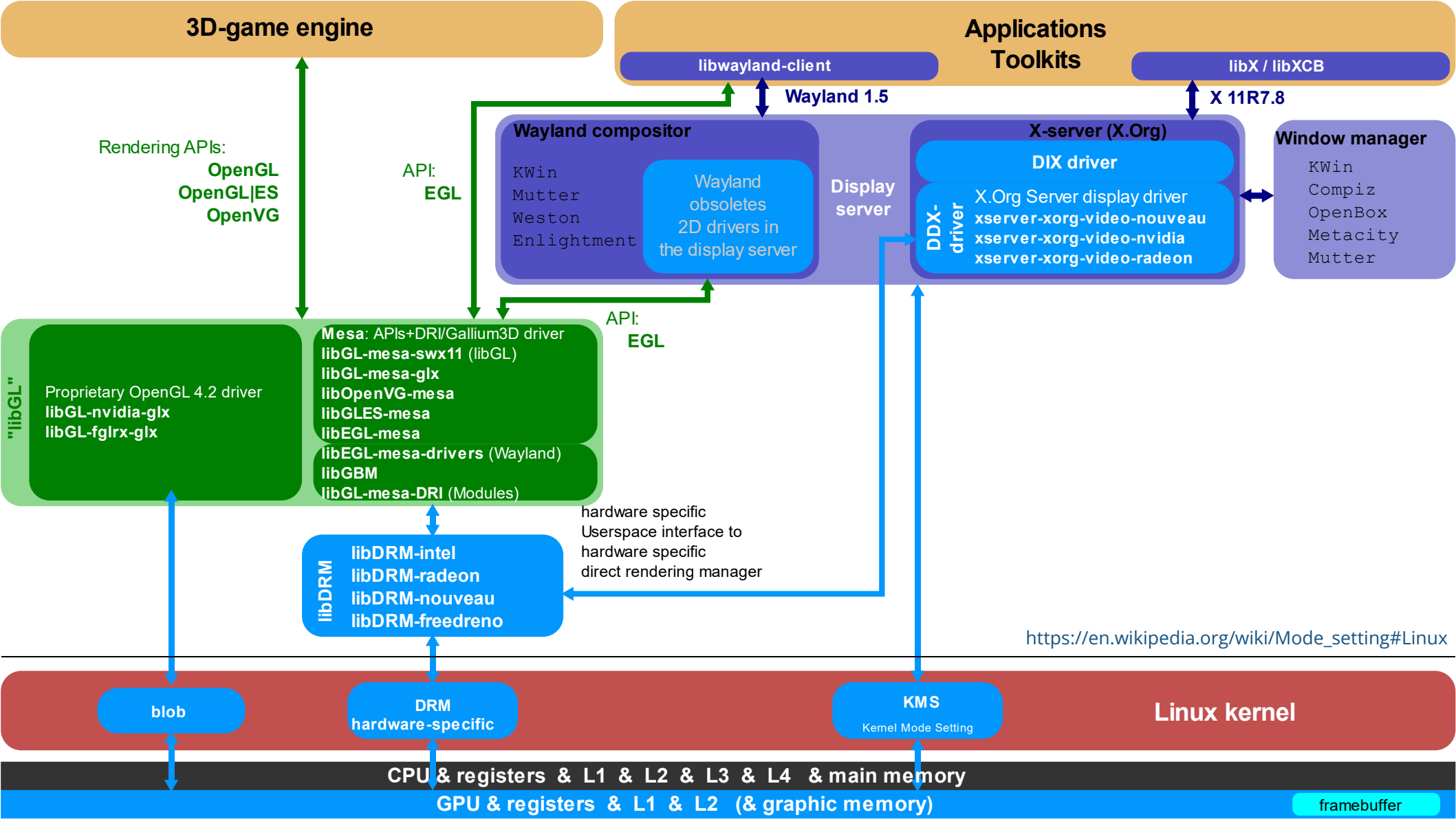


OpenGL Library (libGL.so)

Provides API implementation
– Mesa, NVIDIA, AMD

The Graphics HW/SW Boundary





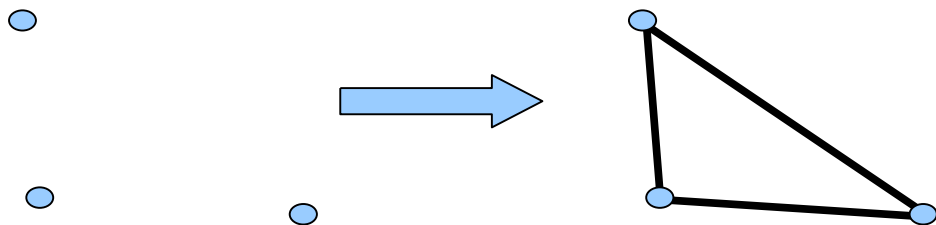
OpenGL Extensions

- OpenGL's capability can be extended on various H/W by providing extensions.
- These have to be loaded dynamically at runtime.
- GL Extension Wrangler (GLEW)
- Helps to figure out what extensions are available to the OpenGL implementation in the system.
- *We use it to test what version and profile of OpenGL the system has.*

OpenGL Basics

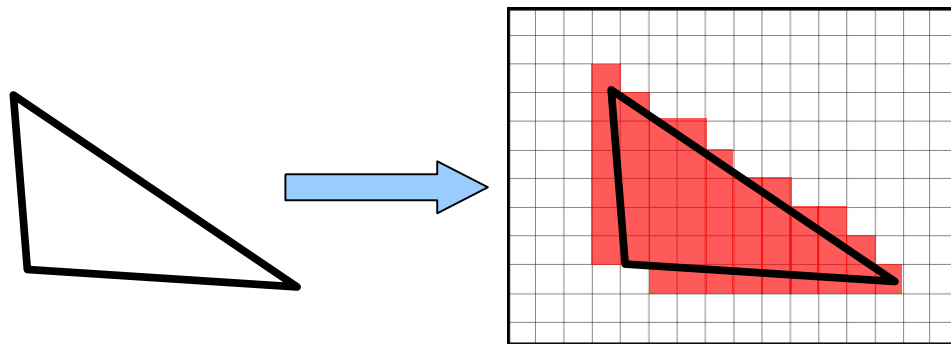
OpenGL Vertices

- A *vertex* is a point coordinate for OpenGL – it is internally represented as a four vector (more on why later).
- Vertices are assembled into *primitives* – Points, Lines, Triangles



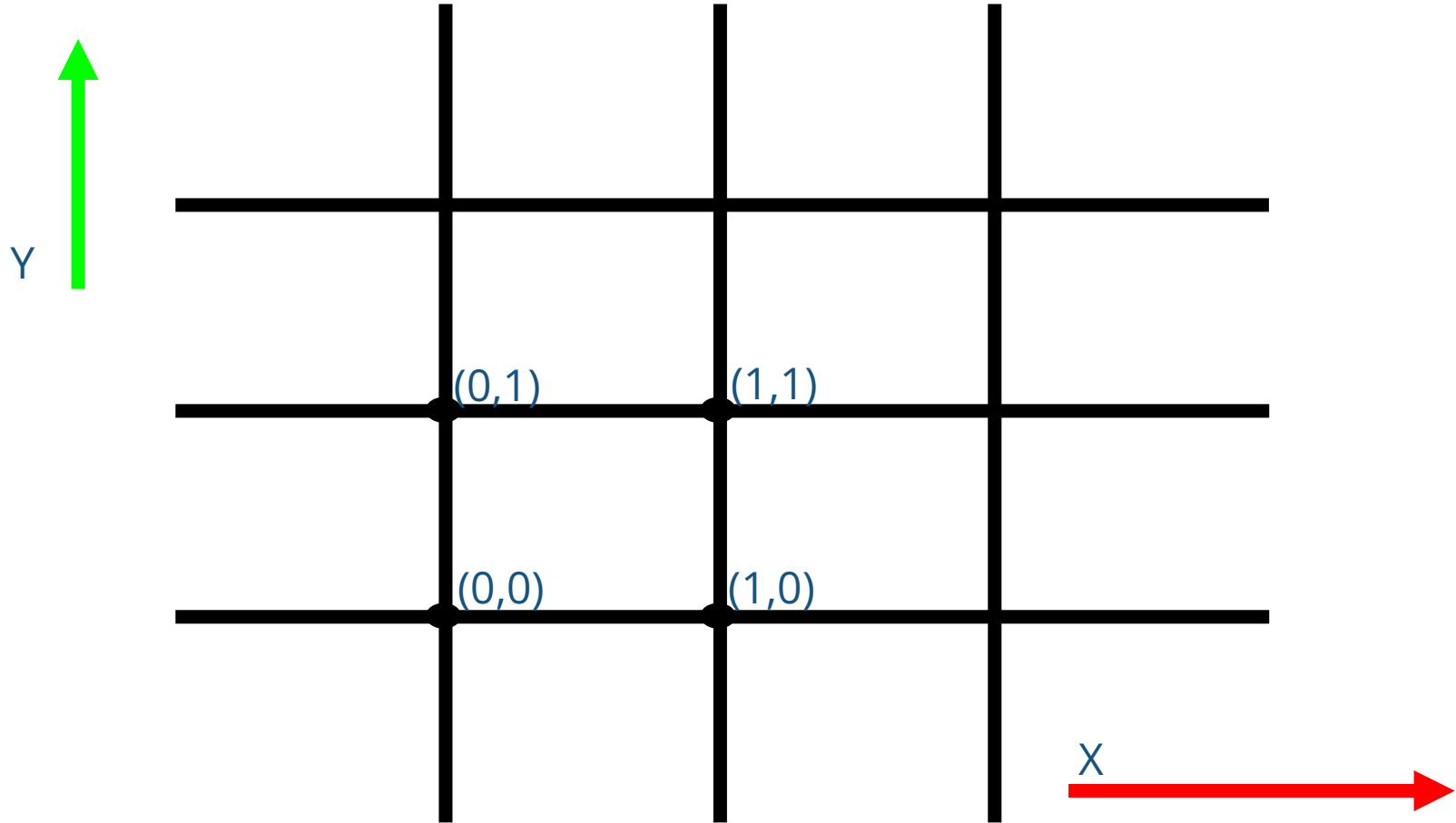
OpenGL Fragments

- A *fragment* is a pixel with a lot of other information:
 - Location
 - Color
 - Normal
 - Depth
 - Opacity
 -

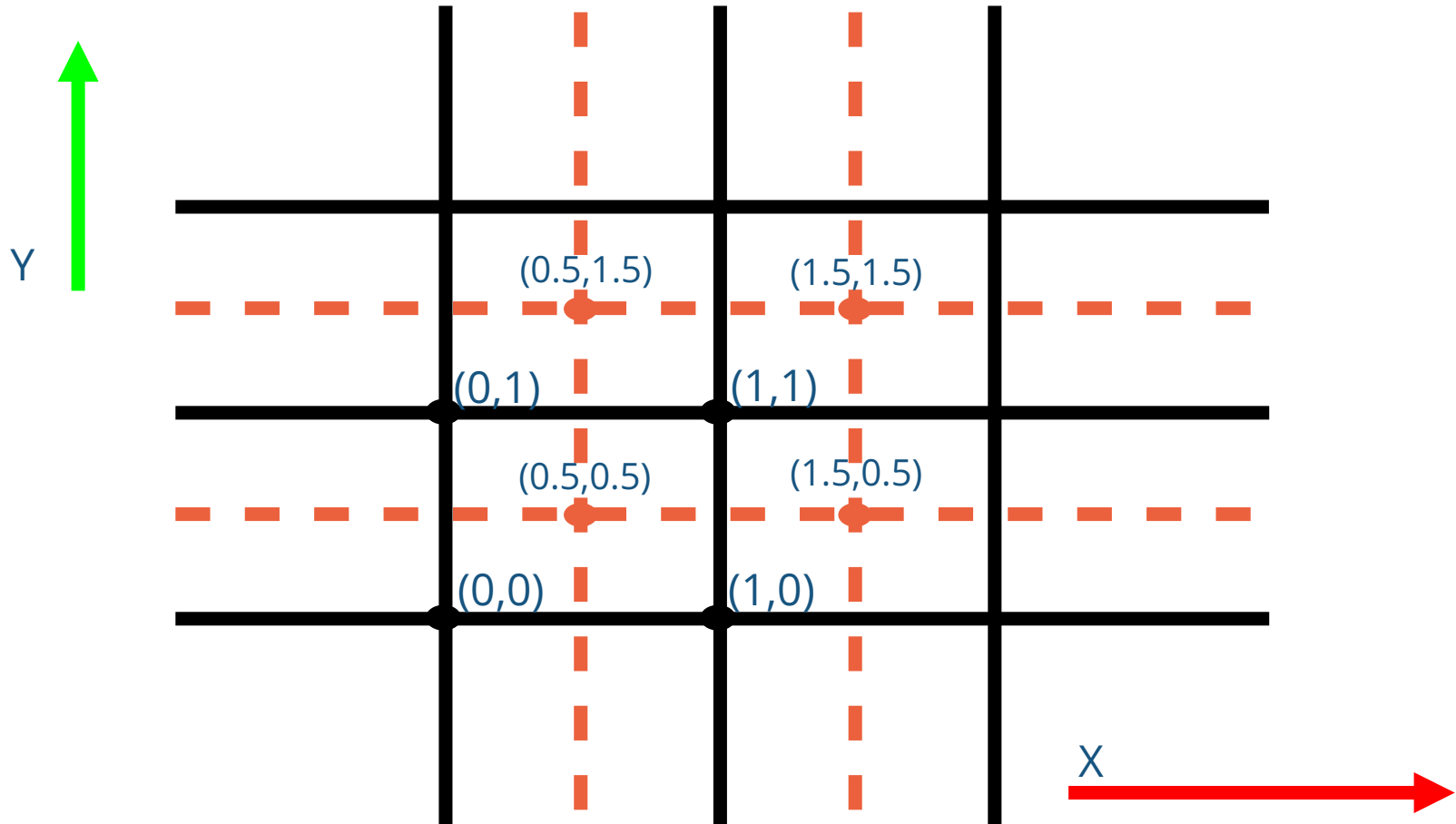


OpenGL rasterizes primitive shapes and outputs fragments.

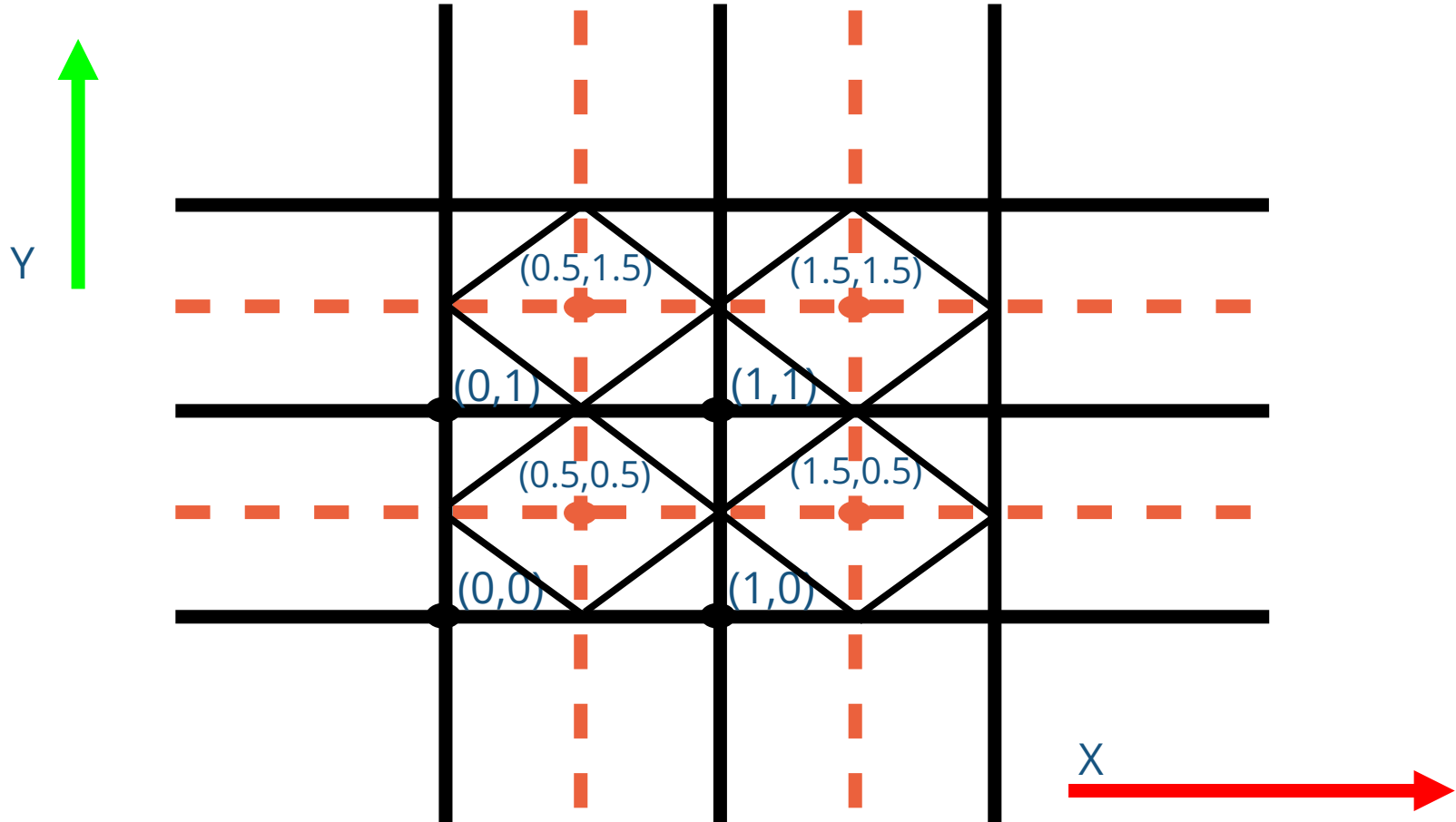
OpenGL Rasterization



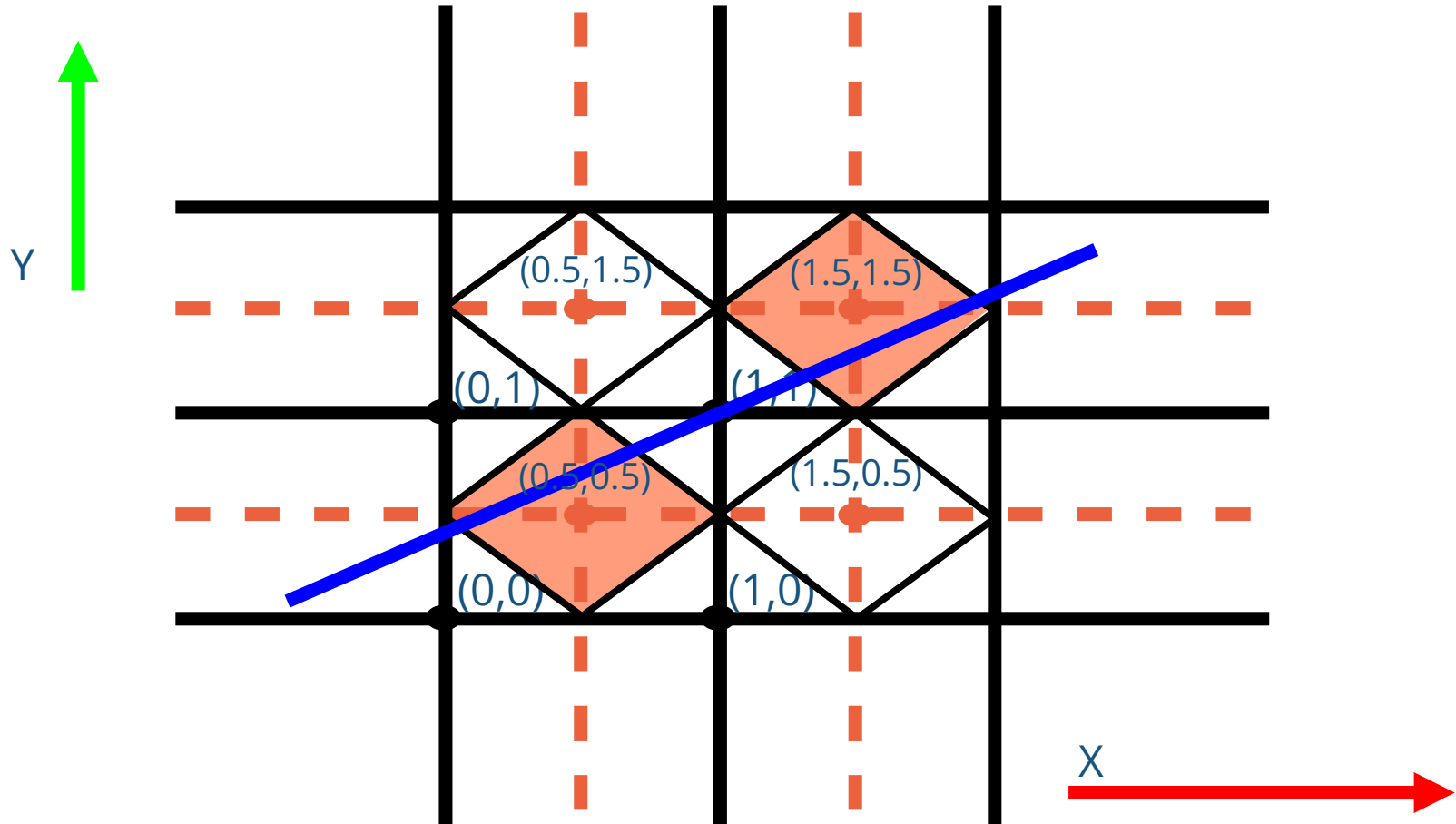
OpenGL Rasterization



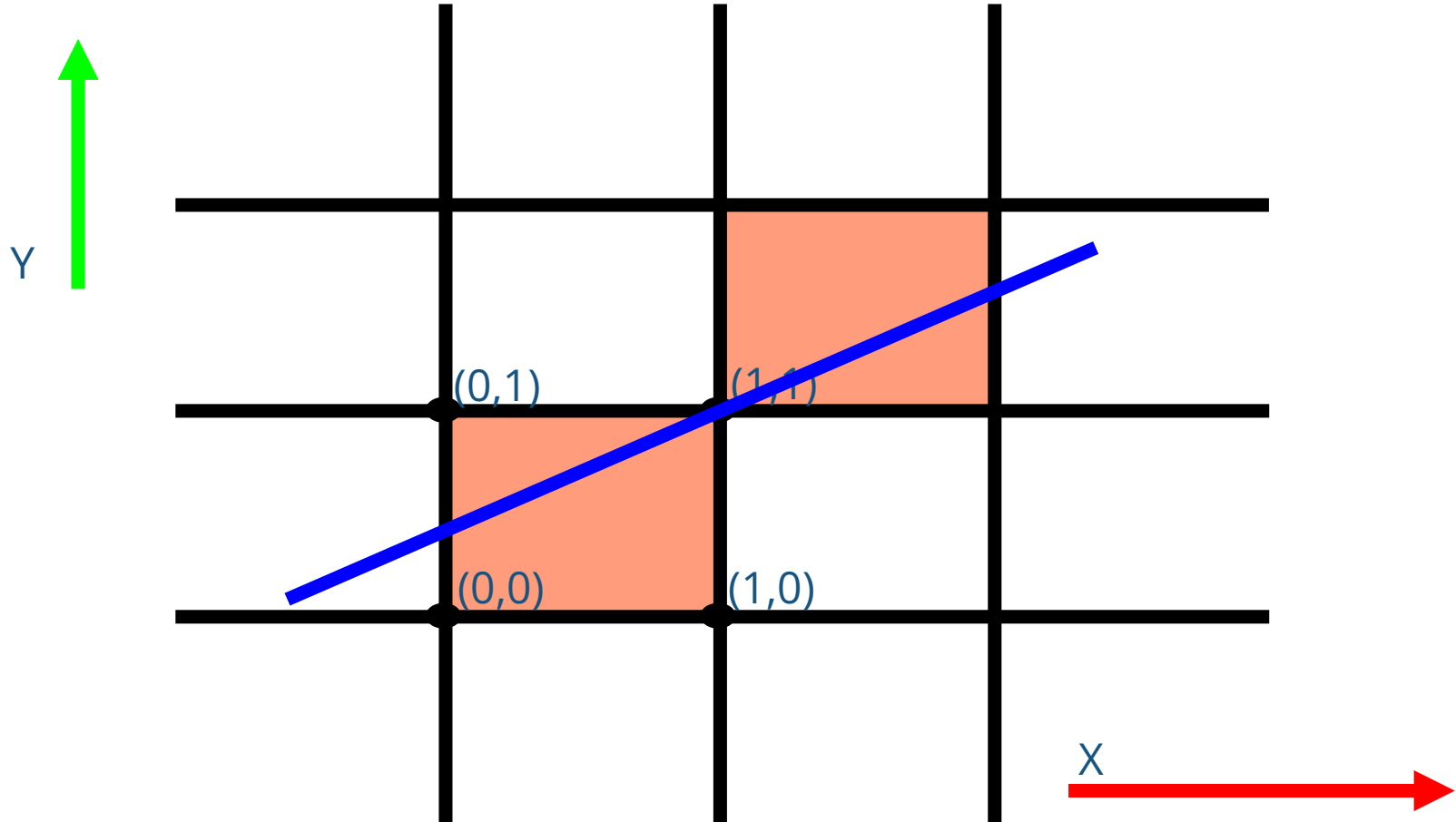
OpenGL Line Rasterization



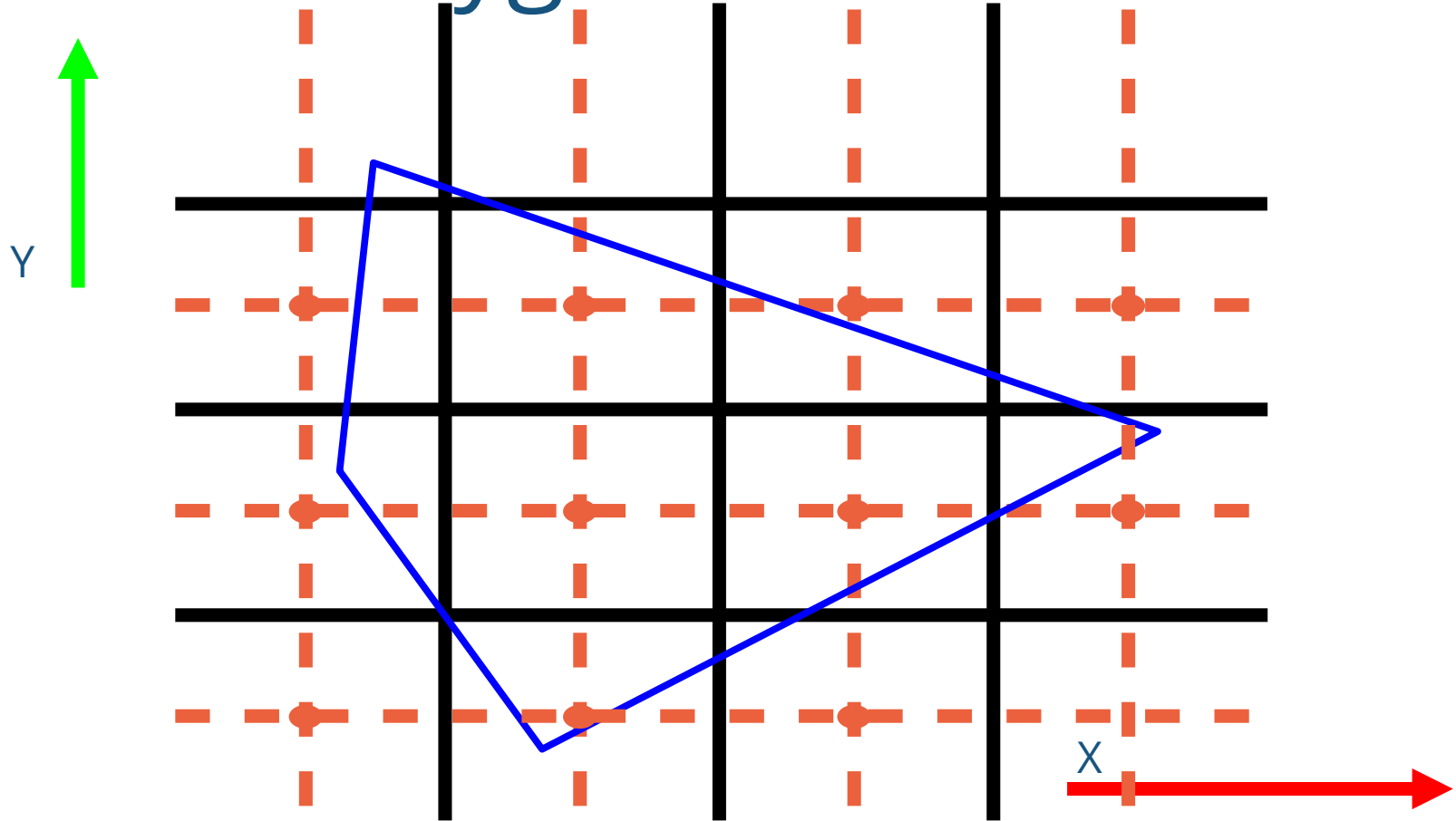
OpenGL Line Rasterization



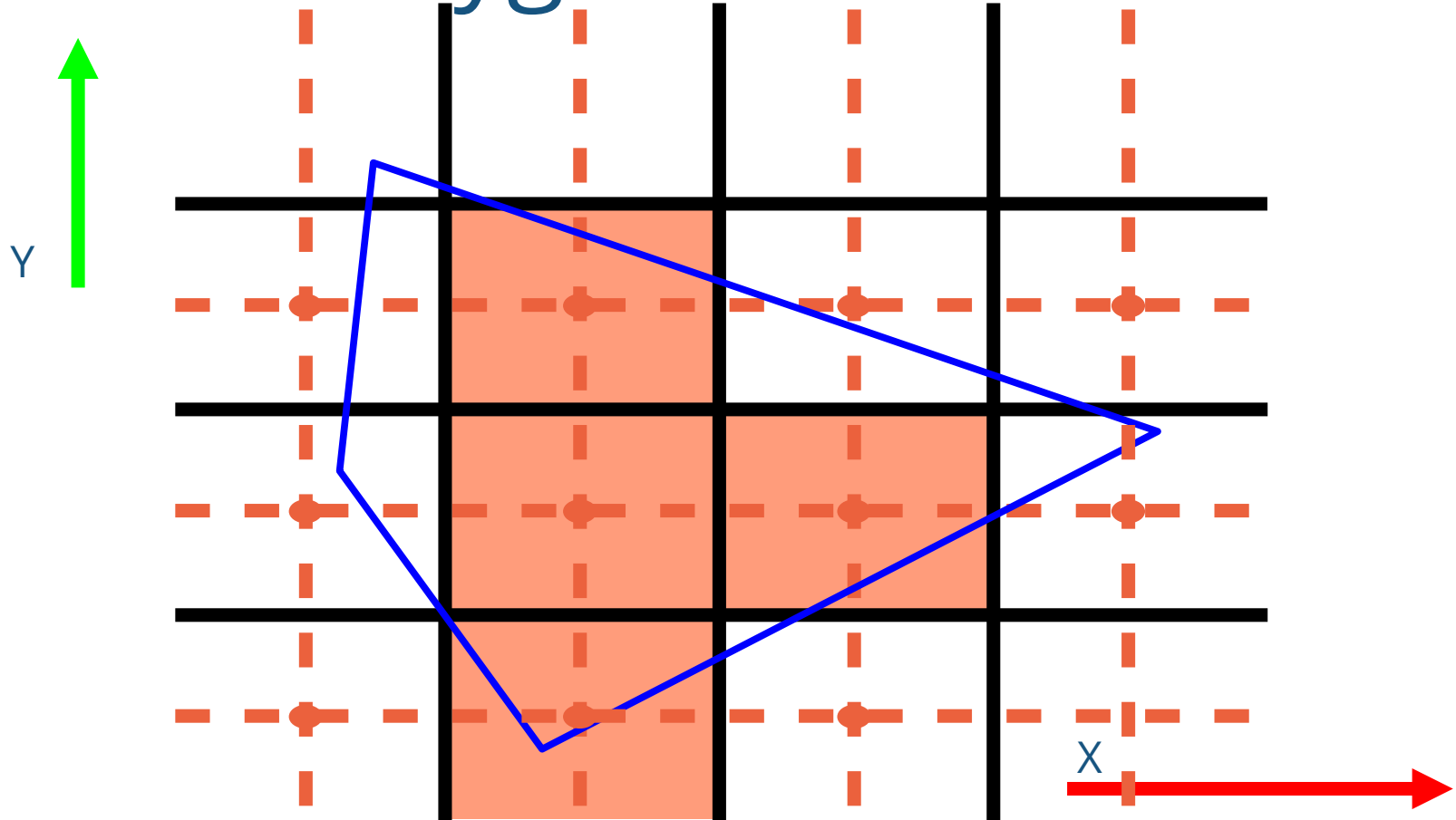
OpenGL Line Rasterization



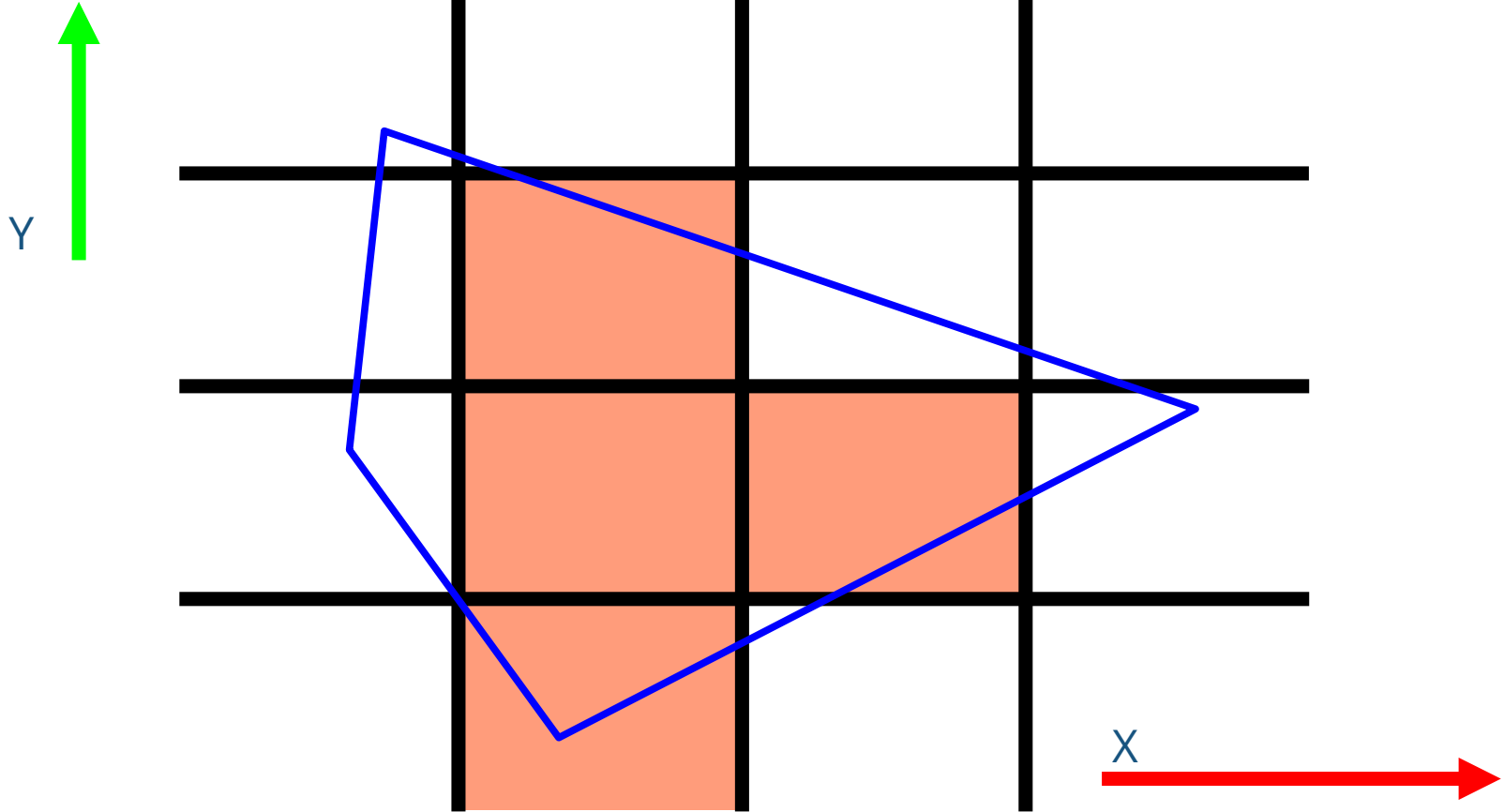
OpenGL Polygon Rasterization



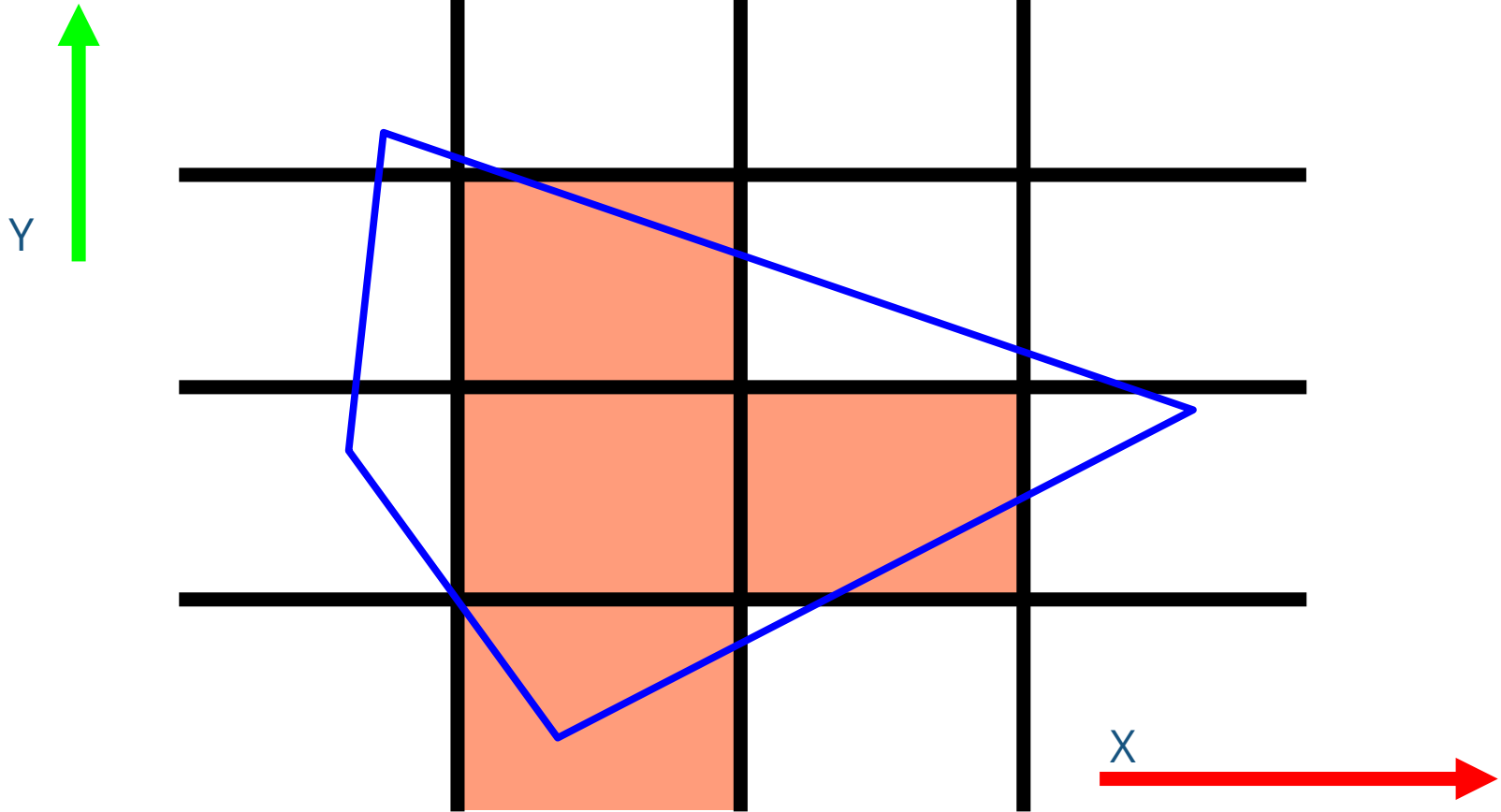
OpenGL Polygon Rasterization



OpenGL Polygon Rasterization

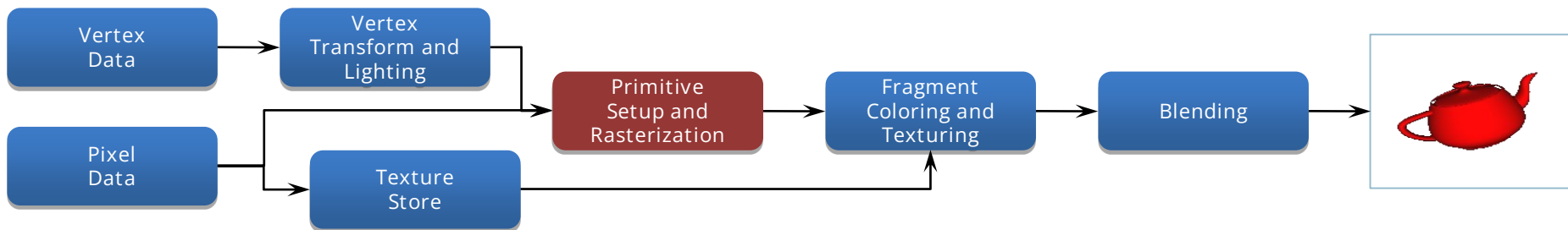


OpenGL Polygon Rasterization



OpenGL 1 (1991)

- Entirely fixed-function

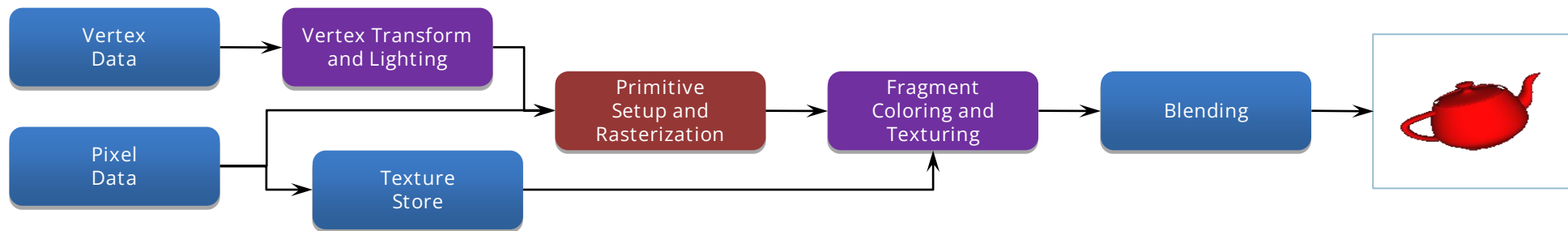


An Introduction to OpenGL Programming, Edward Angel and Dave Schreiner, SIGGRAPH 2013 Course.

- The pipeline evolved but remained based on fixed-function operation through OpenGL versions 1.1 through 2.0.

OpenGL 2 (1994)

- Introduced programmable shaders

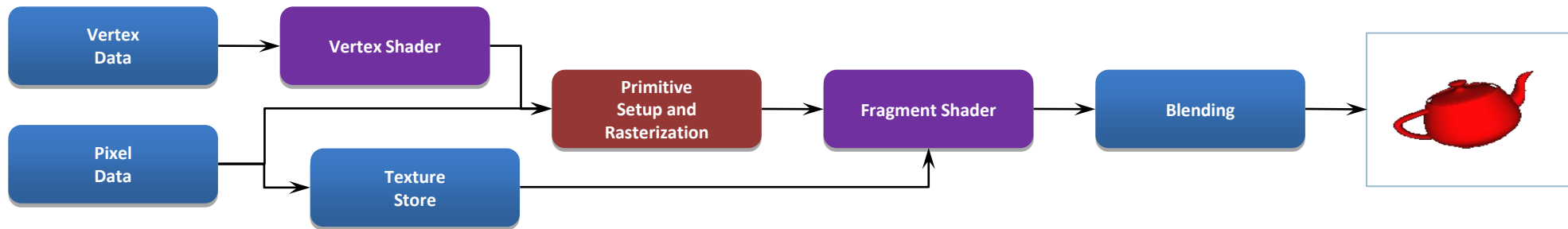


An Introduction to OpenGL Programming, Edward Angel and Dave Schreiner, SIGGRAPH 2013 Course.

- Vertex shading, Fragment shading

OpenGL 3, 3.1, 3.2 (~2008)

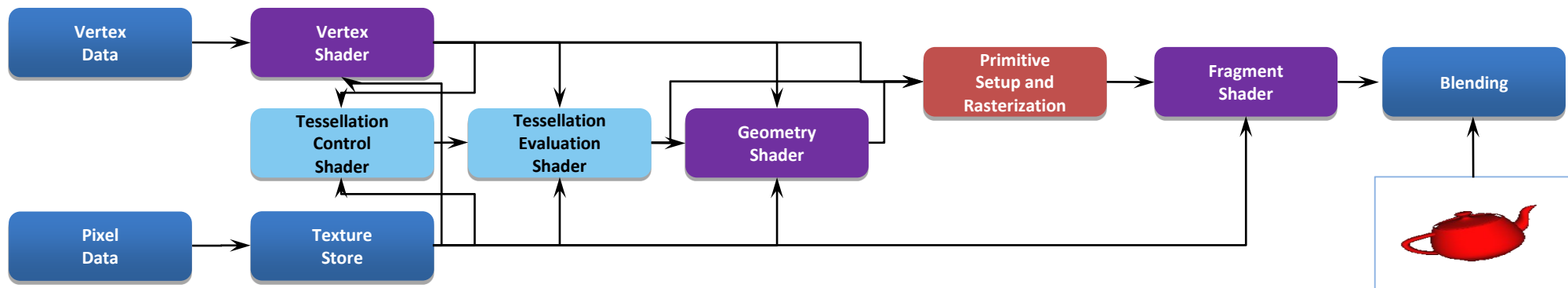
- Introduced the deprecation model
 - Removed the fixed function pipeline.
- Large chunks of data fed to the pipeline instead of small chunks.
- OpenGL catching up with GPU architecture



An Introduction to OpenGL Programming, Edward Angel and Dave Schreiner, SIGGRAPH 2013 Course.

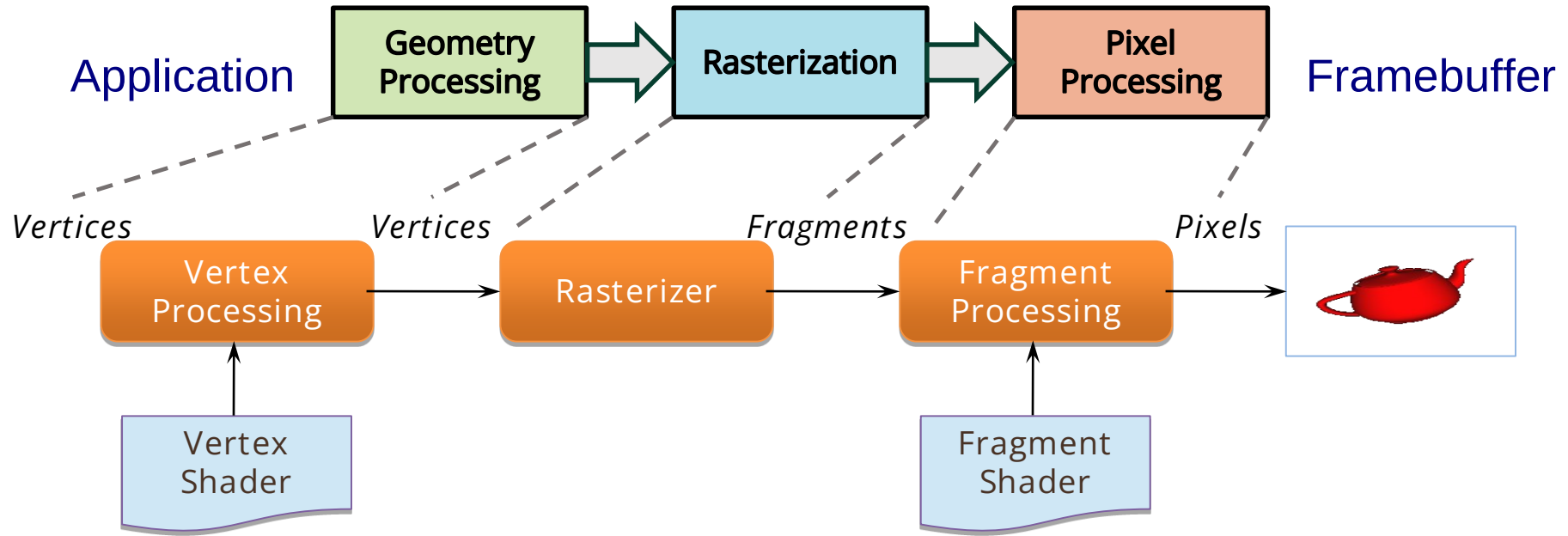
OpenGL 4.0 – 4.6 (current day)

- Geometry & Tessellation Shader – Geometry generated on the GPU

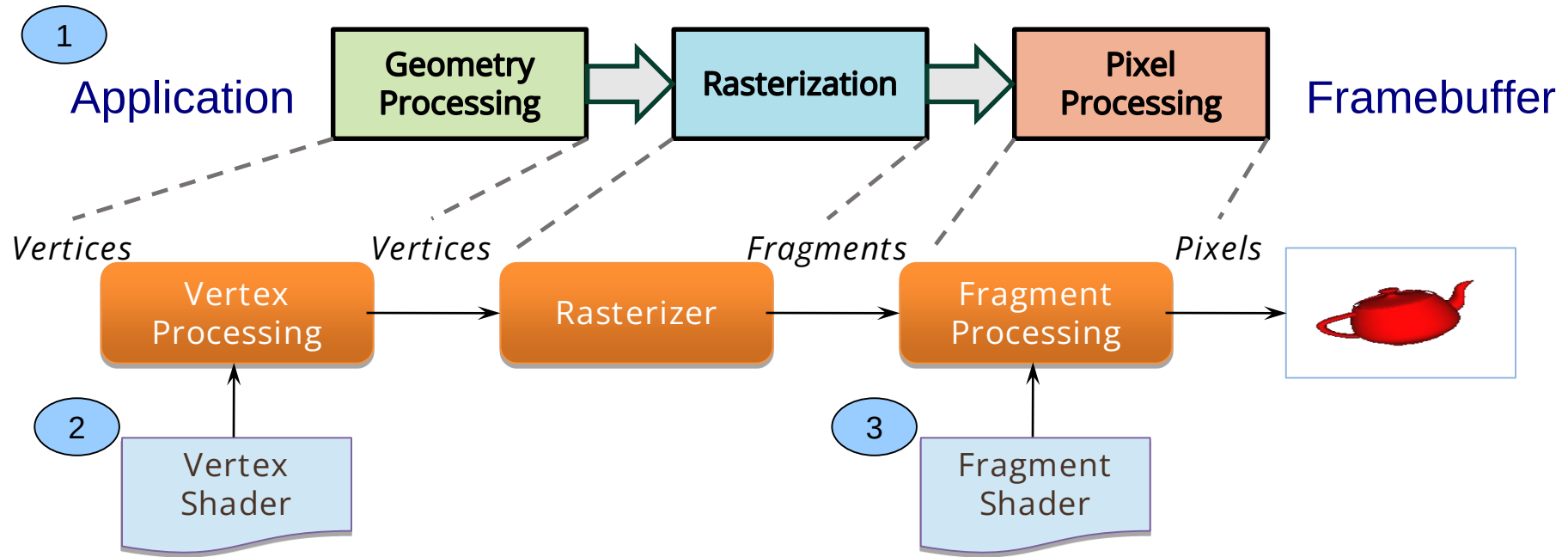


An Introduction to OpenGL Programming, Edward Angel and Dave Schreiner, SIGGRAPH 2013 Course.

Simplified Pipeline

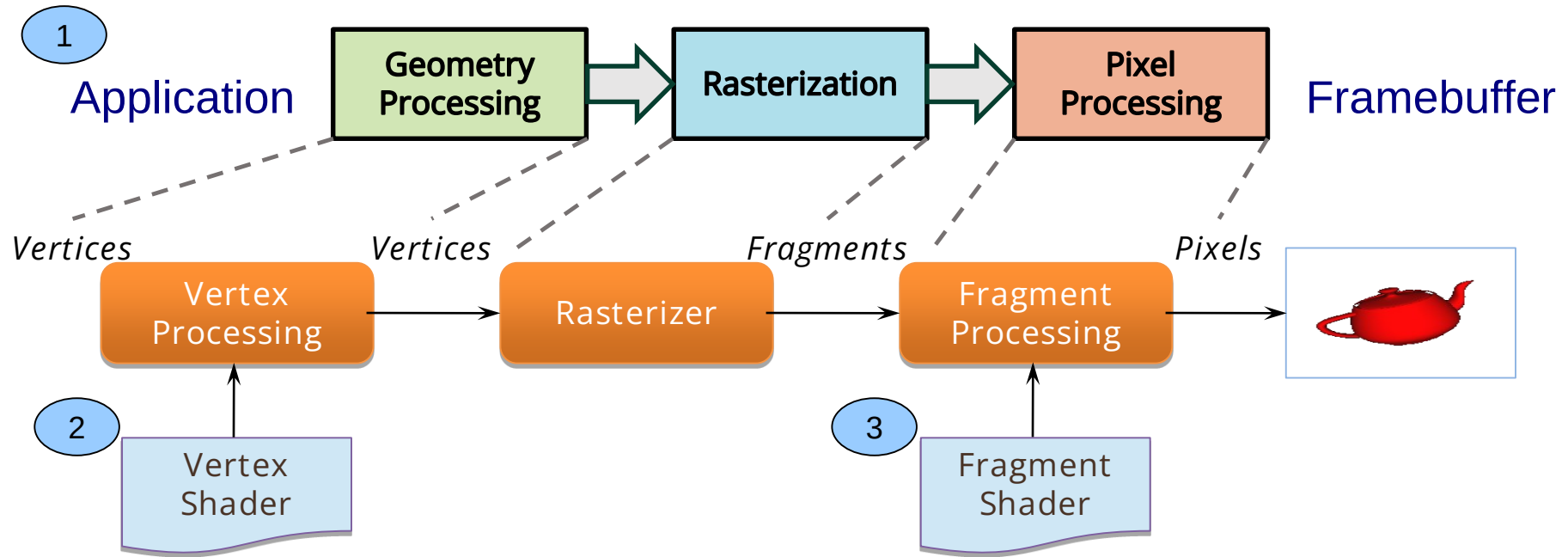


Simplified Pipeline



OpenGL Programming

Simplified Pipeline



OpenGL Application Program

Get geometry data.

Create Buffer Objects and load data into them.

Create Shader Program.

Connect data locations with shader variables.

Windowing system interface: GLFW

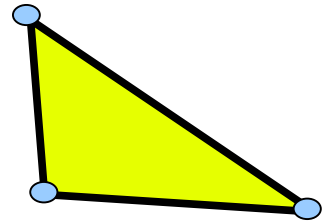
Open windows, Handle Keyboard and Mouse Input

OpenGL Version and Profile handling: GLEW

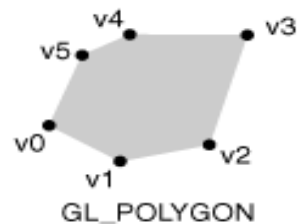
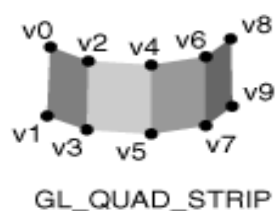
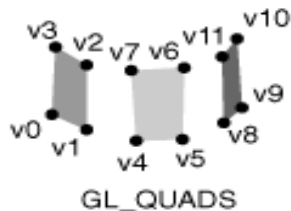
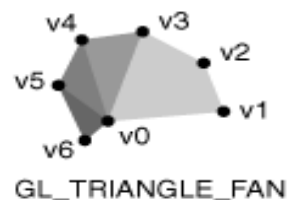
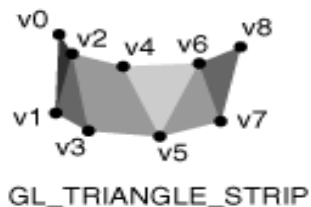
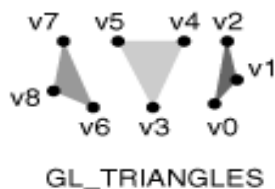
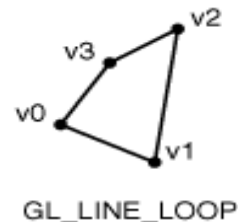
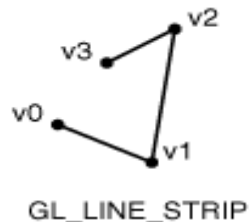
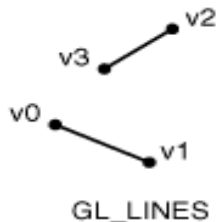
OpenGL Math Library: GLM

Geometric Objects in OpenGL

- A *vertex* is a location in space.
 - Attributes: Position Coordinates
 - Colors, Texture Coordinates, Other Data
- Vertex data must be stored in vertex buffer objects (VBOs)
- Vertex attribute information must be stored in vertex array objects (VAOs)



OpenGL Primitives



Deprecated

Example: ColorCube

A cube with different colours at each vertex

```
//6 faces, 2 triangles/face, 3 vertices/triangles  
const int num_vertices = 36;
```

```
//Eight vertices in homogenous coordinates  
glm::vec4 positions[8] = {  
    glm::vec4(-0.5, -0.5, 0.5, 1.0),  
    glm::vec4(-0.5, 0.5, 0.5, 1.0),  
    ...};
```

```
//RGBA colors  
glm::vec4 colors[8] = {  
    glm::vec4(0.0, 0.0, 0.0, 1.0),  
    glm::vec4(1.0, 0.0, 0.0, 1.0),  
    glm::vec4(1.0, 1.0, 0.0, 1.0),  
    ...};
```

Example: ColorCube

// quad defines two triangles for each face and assigns colors to the vertices

```
void quad(int a, int b, int c, int d)
```

```
{  
    v_colors[tri_idx] = colors[a]; v_positions[tri_idx] = positions[a]; tri_idx++;  
    v_colors[tri_idx] = colors[b]; v_positions[tri_idx] = positions[b]; tri_idx++;  
    v_colors[tri_idx] = colors[c]; v_positions[tri_idx] = positions[c]; tri_idx++;  
    v_colors[tri_idx] = colors[a]; v_positions[tri_idx] = positions[a]; tri_idx++;  
    v_colors[tri_idx] = colors[c]; v_positions[tri_idx] = positions[c]; tri_idx++;  
    v_colors[tri_idx] = colors[d]; v_positions[tri_idx] = positions[d]; tri_idx++;  
}
```

// define 12 triangles: 36 vertices and 36 colors

```
void colorcube(void)
```

```
{  
    quad( 1, 0, 3, 2 );   quad( 2, 3, 7, 6 );  
    quad( 3, 0, 4, 7 );   quad( 6, 5, 1, 2 );  
    quad( 4, 5, 6, 7 );   quad( 5, 4, 0, 1 );  
}
```

Example: ColorCube

Now we load the geometry data into buffers

- VertexArrayObjects (VAOs)
- VertexBufferObjects (VBOs)
- Generate and Bind

Example: ColorCube

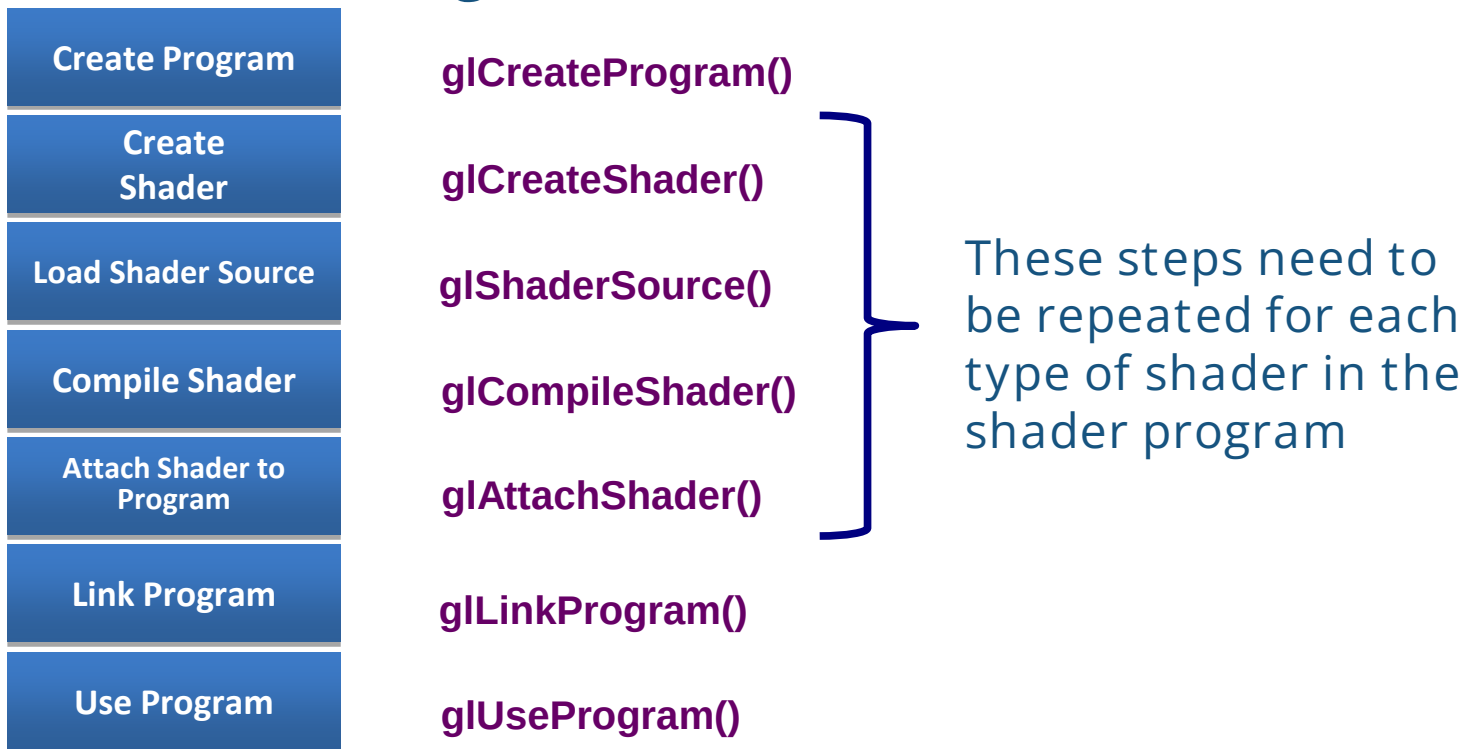
- Vertex Array Object
 - Stores attribute data corresponding to the VBOs
- `GLuint vao;`
- `glGenVertexArrays( 1, &vao );`
- `glBindVertexArray( vao );`

Example: ColorCube

- Vertex Buffer Object
- Vertex data must be stored in a VBO, and associated with a VAO
- The code-flow is similar to configuring a VAO
 - generate VBO names by calling `glGenBuffers()`
 - Bind a specific VBO for initialization by calling `glBindBuffer( GL_ARRAY_BUFFER, ... )`
 - Load data into VBO using `glBufferData( GL_ARRAY_BUFFER, ...)`
- `glBufferSubData( GL_ARRAY_BUFFER, ... )`

Example: ColorCube

Next we create a Shader Program



Example: ColorCube

GLSL Vertex Shader

```
#version 430
```

```
in vec4 vPosition;  
in vec4 vColor;
```

```
out vec4 color;
```

```
void main ()  
{  
    gl_Position = vPosition;  
    color = vColor;  
}
```

Example: ColorCube

GLSL Fragment Shader

```
#version 430
```

```
in vec4 color;
```

```
out vec4 frag_color;
```

```
void main ()
```

```
{  
    frag_color = color;  
}
```

Example: ColorCube

- Connect application program data to shader variable.
- OpenGL relates shader variables to indices for the application to set
- Have to find variable/index association
 - Before linkage
 - After linkage

Example: ColorCube

- Connect application program data to shader variable.
- Assumes shader variable names are known

```
GLint loc_idx = glGetAttribLocation( program_id, "variable_name" );
```

```
GLint loc_idx = glGetUniformLocation( program_id, "variable_name" );
```

```
glEnableVertexAttribArray( loc_idx );
```

```
glVertexAttribPointer( loc_idx, 4, GL_FLOAT, GL_FALSE, 0,  
                      pointer_to_data_in_buffer );
```

Example: ColorCube

- Draw

```
glDrawArrays(GL_TRIANGLES, 0, num_vertices);
```

- Calls a vertex shader for each vertex.
- Assembled into triangles and rasterized to fragments.
- Calls a fragment shader for each fragment.

Example: ColorCube

GLFW

Open a window.

Initialize an
OpenGL context
in it.

```
GLFWwindow* window;  
glfwSetErrorCallback(csX75::error_callback);  
  
if (!glfwInit()) return -1;  
  
//We want OpenGL 3.3  
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);  
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);  
  
//We don't want the old OpenGL  
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);  
  
//! Create a windowed mode window and its OpenGL context  
window = glfwCreateWindow(512, 512, "CS475/CS675 Tutorial 2: Colorcube", NULL,  
NULL);  
if (!window) { glfwTerminate(); return -1; }  
  
//! Make the window's context current  
glfwMakeContextCurrent(window);
```

Example: ColorCube

GLEW

Check available
OpenGL
capabilities.

```
/Initialize GLEW
//Turn this on to get Shader based OpenGL
glewExperimental = GL_TRUE;
GLenum err = glewInit();
if (GLEW_OK != err)
{
    //Problem: glewInit failed, something is seriously wrong.
    std::cerr<<"GLEW Init Failed : %s"<<std::endl;
}
```

Example: ColorCube

GLFW

Register
callbacks

```
glfwSetKeyCallback(window, csX75::key_callback);  
glfwSetFramebufferSizeCallback(window, csX75::framebuffer_size_callback);
```

```
glfwSetInputMode(window, GLFW_STICKY_KEYS, GL_TRUE);
```

Start the render
loop.

```
//Initialize GL state  
csX75::initGL(); initBuffersGL();  
  
// Loop until the user closes the window  
while (glfwWindowShouldClose(window) == 0)  
{  
    renderGL();  
    glfwSwapBuffers(window);  
    glfwPollEvents();  
}  
  
glfwTerminate();  
return 0;  
}
```

Example: ColorCube

